

Lab 3: Architectural Pattern Selection & Justification

Database Query Performance Profiler

Yajat Mathur | PES2UG24CS606

Architecture Selection: We chose the Microservices Architecture for the Database Query Performance Profiler System.

Reason 1: Independent Scaling of Ingestion vs. Analysis

The Log Ingestion Service must continuously absorb up to 5,000 slow-query log records per minute (NFR-001) without slowing the host database, while the Query Analysis Service performs comparatively heavier EXPLAIN-plan parsing and index recommendation (FR-001). A microservices split lets each service be scaled independently — the ingestion pipeline can be replicated horizontally during peak query load without over-provisioning the analysis engine, and vice-versa. A monolithic layered design would force both workloads to scale together, wasting resources.

Reason 2: Fault Isolation for Time-Critical Alerting

Critical-latency alerts to the Backend Lead must be dispatched within 30 seconds of a regression (FR-005), and the weekly digest (FR-003) and dashboard must remain independently available even if one module fails. Because each capability — ingestion, analysis, alerting, digesting, and configuration — is deployed as its own service, a failure or slow restart in the Digest Reporting Service, for example, cannot block real-time alert delivery or ongoing log ingestion. This isolation is difficult to achieve in a tightly-coupled layered or client-server design.

Security Advantage

NFR-002 requires all ingested log data and execution plans to be encrypted in transit (TLS 1.3) and at rest (AES-256). With a microservices architecture, only the Log Ingestion Service and Metrics & Recommendation Store need direct, credentialed access to the target database and encrypted storage volumes; the Dashboard UI, Alert Notification, and Digest Reporting services never touch raw connection secrets. This narrows the blast radius of a compromised service and lets each service enforce TLS termination and encryption independently at its own boundary, rather than relying on a single shared security perimeter.

Performance Benefit

Decoupling the high-throughput ingestion path from the CPU-intensive EXPLAIN-plan analysis and the scheduled weekly digest generation (FR-003) prevents a burst of incoming slow-query logs from delaying critical alert delivery or dashboard responsiveness. Each service can be deployed with resource limits and caching tuned to its own workload profile, and the asynchronous LogStream and CriticalLatencyEvent interfaces between services (see Component Diagram) let ingestion continue uninterrupted even while analysis or alerting is briefly under load, keeping end-to-end alert latency within the 30-second target.